TECHNISCHE HOCHSCHULE NÜRNBERG
GEORG SIMON OHM

Fakultät Elektrotechnik Feinwerktechnik Informatik

Conceptual design and realization of a graphical tool for determining the test coverage of digital circuit diagrams

Masterarbeit im Studiengang Softwareengineering und Informationstechnik

vorgelegt von

Razvan Matis

Matrikelnummer 2403005

Erstgutachter:	Prof. Dr. Flaviu Popp-Nowak
Zweitgutachter:	Prof. Dr. Claus Kuntzsch

Abstract

According to Printed circuit board (PCB)s and their digital circuits which are getting more complex all the time there is the requirement of being able to determine the test coverage of it. In dependency on these determined values then automatic boundary scan chains could be performed or the need of changing something on the layout will be visible. This document describes the way of the conceptual design and the realization of a graphical tool for loading, showing different digital circuit diagrams projects which need to be in the Open database (ODB)++ format and the determination of test coverage parameters as well as the creation of Serial vector format (SVF) files for being able to run the boundary scan chain on these special devices which contains at least one Joint test action group (JTAG) device. Furthermore there will be described detailed information about the frameworks, programming patterns and best practices which were used for creating this Model-view-viewModel (MVVM) C# project and the project structure itself which was created for making sure to have the following key features available:

- Opening any ODB++ project by connecting to a stand alone Google remote procedure calls (GRPC) server and using the PCB-Investigator API
- Exporting any opened projects to Java script object notation (JSON) file for later usage and importing of that created files
- Creation and loading of project files which contains a manifest file for information about the project structure itself
- Modifying, exporting and importing of different kind of settings according the connection to the GRPC server, the appearance and the test coverage values
- Determination of different test coverage values according to the boundary scan mechanism
- Creation of SVF files for run the boundary scan chain with a programmer from ProMik GmbH on a JTAG containing device

Contents

Abstract . . . . . **ii**

1. List of abbreviations **1**

2. Introduction **2**
2.1. Motivation 2
2.2. Requirements 2
2.3. Programming language and operating system 4

3. Fundamentals **5**
3.1. ODB++ project format 5
3.2. PCB-Investigator 6
3.3. JTAG interface 7
3.4. Boundary scan 9
3.5. Boundary scan description language 10
3.6. SVF file format 11
3.7. Model-view-viewModel 11
3.8. PRISM framework 12
3.9. Dependency injection 13
3.10. Material design framework 13
3.11. JSON format 13
3.12. GRPC 14

4. Conceptual design **16**
4.1. Programming language to use 16
4.2. Integrated development environment to choose 16
4.3. Application structure to consider 17
4.4. Target frameworks to use 18
4.5. Analyzing the structure of PCB-Investigator API 19
4.6. Converted class structures of objects of interest 20
4.7. Frameworks and patterns to use 21

5. Architecture **23**
5.1. Overview 24
5.2. Interfaces 24

5.3. Grpc Server for using the PCB Investigator API . . . 24
5.4. Grpc Client for connecting to the Server and parsing all objects . . . 24
5.5. GUI . . . 24
5.5.1. Events . . . 24
5.5.2. Helper . . . 24
5.5.3. Interfaces . . . 24
5.5.4. Implementations . . . 24
5.5.5. Model . . . 24
5.5.6. ViewModels . . . 24
5.5.7. Views . . . 24
5.5.8. MainView and MainViewModel . . . 24
5.6. Test coverage determination of loaded project . . . 24
5.7. Pin information extraction . . . 24
5.8. SVF file generator . . . 24
5.9. Unit tests . . . 24

6. Analysis . . . 25
6.1. Performance of loading ODB++ projects by using the PCBInvestigator API 25
6.2. Performance of loading projects by importing JSON formatted files . . . 25
6.3. Performance of exporting projects into JSON format . . . 25
6.4. Performance of showing connections of components . . . 25
6.5. Performance of determination of test coverage values . . . 25

7. Conclusion . . . 26

8. Thanksgiving . . . 27

A. Appendix . . . 29

List of Figures . . . 30

List of Tables . . . 31

List of Listings . . . 32

Bibliography . . . 33

Chapter 1.

List of abbreviations

PCB Printed circuit board

ODB Open database

JTAG Joint test action group

SVF Serial vector format

GRPC Google remote procedure calls

JSON Java script object notation

MVVM Model-view-viewModel

MCU Micro controller

GUI Graphical user interface

CPU Central processing unit

RAM Random access memory

GB Gigabyte

CAD Computer-aided design

API Application interface

BSDL Boundary scan description language

WPF Windows presentation foundation

XAML Extensible application markup language

IDE Integrated development environment

Chapter 2.

Introduction

Within this chapter the motivation will be described, the requirements, the used programming language and operating system for creation of the whole project. All these aspects where previously discussed with the company to make sure to create a most valuable software for them.

2.1. Motivation

As the company ProMik GmbH is already providing programmers for being able to write date into different Micro controller (MCU)s now there came up the requirement, in dependency on customer demands, to perform boundary scan test chains on JTAG devices. For being able to perform these kind of tests there should be a software available which allows to determine first the test coverage parameters in dependency on the boundary scan mechanism and create such SVF files for playing them with the programmer. This software should be provided as a complex framework where the opportunity is given to extend it in the future in elementary way by using much interfaces within the MVVM pattern.

2.2. Requirements

In order to provide all features as needed there were a couple of requirements which popped up at the very beginning of the conceptual design and during the implementation of the software:

- There should be the possibility of opening ODB++ project folders and display the result of circuit diagram in a useful way
  - There should be the possibility of moving the visible area by mouse movement and zooming in and out of the visible area

  - The visibility of each component should be settable
  - All connections of one component to the others should be displayable
  - The whole view of all components could be mirrorable along both axes
  - The net names should be settable to visible and hidden
- An export mechanism of an opened project into JSON file for later usage should be available as an according import mechanism to be independent from the PCB-Investigator API as this requires a separate license for usage
- There should be the option of creating and loading zip compressed project files containing all necessary information
  - If the project file mode is being used then all exports and opening of components should be done and saved within that project file
  - If the project file contains settings, a JSON exported project then these files should be imported automatically on loading
- Settings should be available, exportable and importable
  - General settings according the appearance of the loaded project view
  - GRPC-Server related connection settings
  - General settings according the request of PCB-Investigator API
  - Test coverage related settings for determining all components for the boundary scan chain
  - SVF related settings according the programmer connection parameters to use
- Determination of test coverage parameters using the boundary scan mechanism
- Export of a file containing all pin information of all JTAG devices
- Creation of SVF files containing all instructions to be performed by the programmer while playing the boundary scan test chain on a JTAG compatible device
- In order to create an extended-friendly framework there was furthermore the need of usage of company internal libraries instead of creating complete new ones
  - The usage of log panel was required
  - The usage of setting panel was required
  - The usage of SVF player was required

2.3. Programming language and operating system

The whole project was implemented with the software development environment called **Microsoft Visual Studio 2019** and the programming language **C#**. This is a quit powerful and modern language for creation of different desktop applications. The developer environment is really comfortable too, as it provides a NuGet package manager for organizing all external included libraries. Furthermore the whole project was created on a operating system **Windows 10 Professional x64** using a Central processing unit (CPU) of type **Intel(R)Xeon(R) E3-1240 v6 @ 3.70 GHz** and a total amount of Random access memory (RAM) of **16 Gigabyte (GB)**. This system makes sure to be able to implement without any delays of waiting for background processes at all and ensures to be able to run the developed software almost immediately without any delay times.

Chapter 3.

Fundamentals

Within this chapter the fundamentals regarding the later usage for implementation of the project is being described.

3.1. ODB++ project format

The ODB++ format is a de facto standard format for creation of Computer-aided design (CAD) related projects[1] within the circuit board industry. This format itself was released in 1995 and after the component names were added then the suffix ++ was added in 1997. It hierarchically contains all necessary information about the digital circuit diagram in detail:

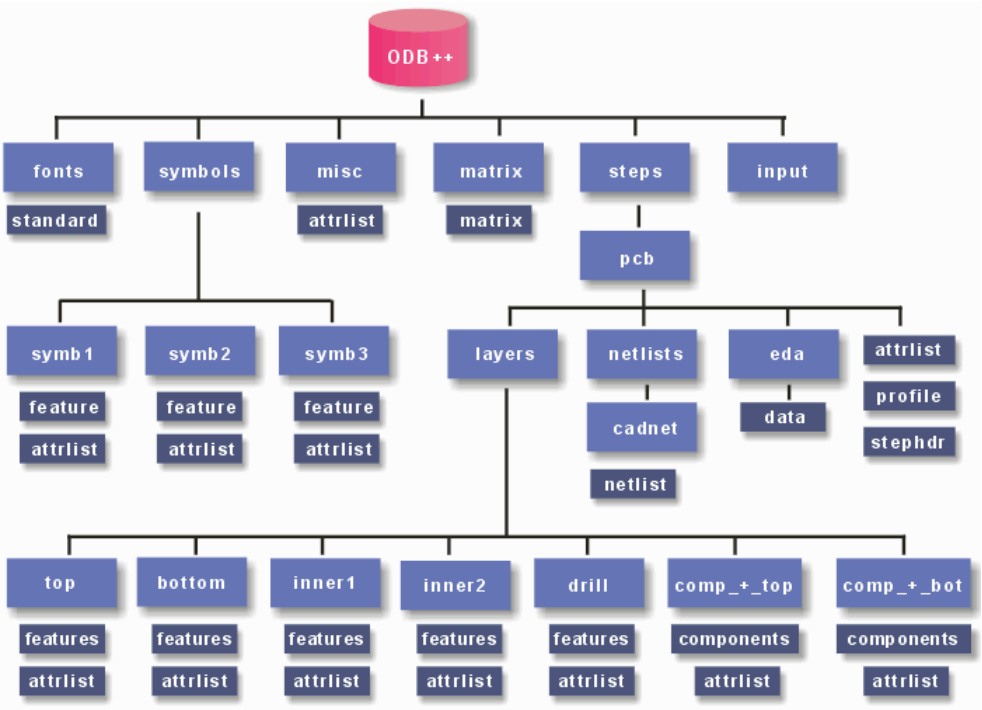

Figure 3.1.: ODB++ project structure[2]

As shown in 3.1 it is clear that this format contains all necessary components, net lists and attribute lists which are necessary for later determination of the test coverage of the circuit diagrams. These data will give detailed information regarding the placed components, their names, positions, sizes and connection to all nets. In dependency on the net connection the connected components could then be determined

3.2. PCB-Investigator

The PCB-Investigator is a software provided by EasyLogix also known as Schindler & Schill GmbH and provides many functionality regarding the opening, viewing and measuring of different kind of CAD related project files of digital circuit diagrams[3]:

- ODB++
- Gerber 274x
- Gen Cad 1.4
- Eagle files
- IDF files
- DXF files
- DPF files
- IPC2581 files
- IPC356 files
- PDF

It is a quit powerful tool which further provides the possibility to export almost all project types into another one and generate different kind of lists containing component relevant information as the component name and its pin information. Further the executable file application itself provides different public functions which are accessible by adding the **PCB-Investigator.exe** as a reference to the project.

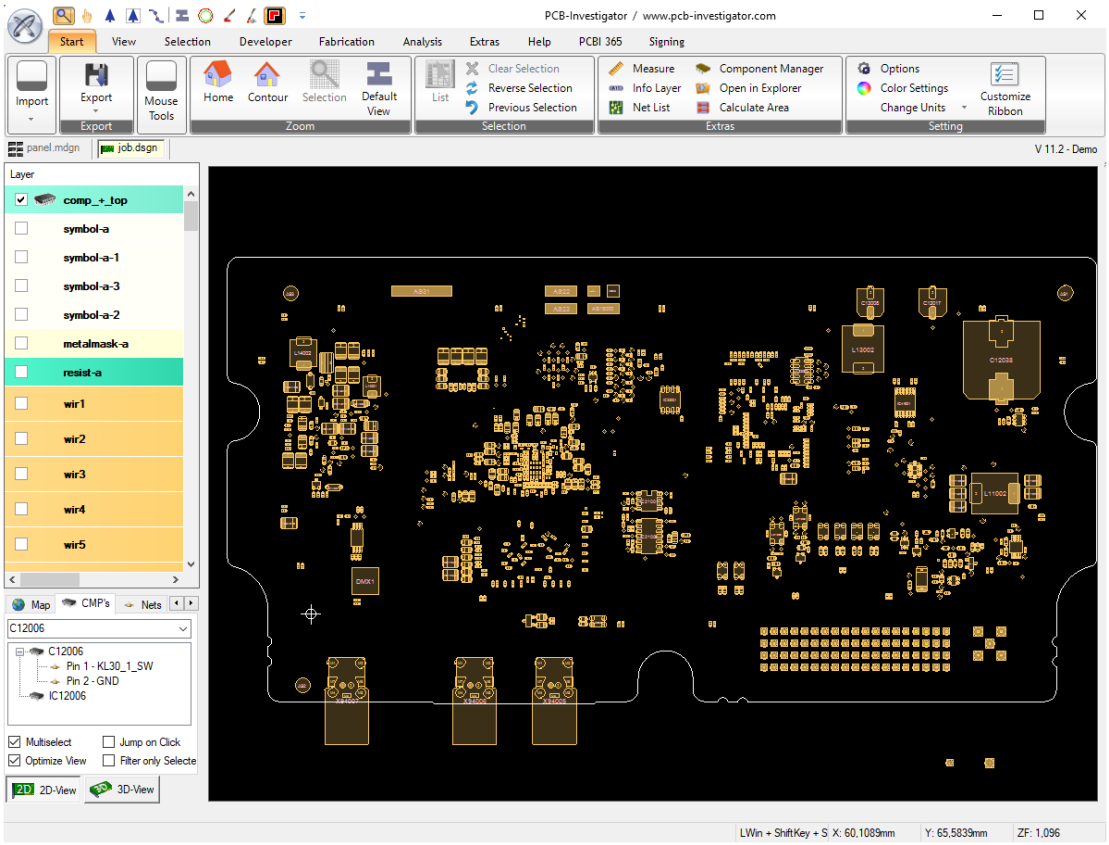

Figure 3.2.: PCB-Investigator overview

As pictured in 3.2 it can be seen that there are many possibilities of taking action with a loaded project. For verification purposes the most important feature was the CMPs tab where it is possible to search for any specific component in dependency on its name and to have a closer look in its consisting pins.

3.3. JTAG interface

The JTAG is a serial interface which is widely used for different field programmable gate arrays, complex programmable logic device configurations, the programming and debugging of different micro controllers. It is usually operating within the voltage range of 1.8V - 6V and consists of the following components in general:

- Test access port
  - Pin TDI - data in

  - Pin TDO - data out
  - Pin TMS - control
  - Pin TCK - clock
  - Pin TRST - test reset
- Instruction register
- Data register
- TAP controller

This standard established within 1985/1986 and is being normed by IEEE 1149.11990. Later on by the overworked norm IEEE 1149.11994 the Boundary scan description language (BSDL) was considered as a default feature. The most actual version of the norm is IEEE 1149.1-2001. If there are more TAPs connected together then it will be called a scan chain in general.

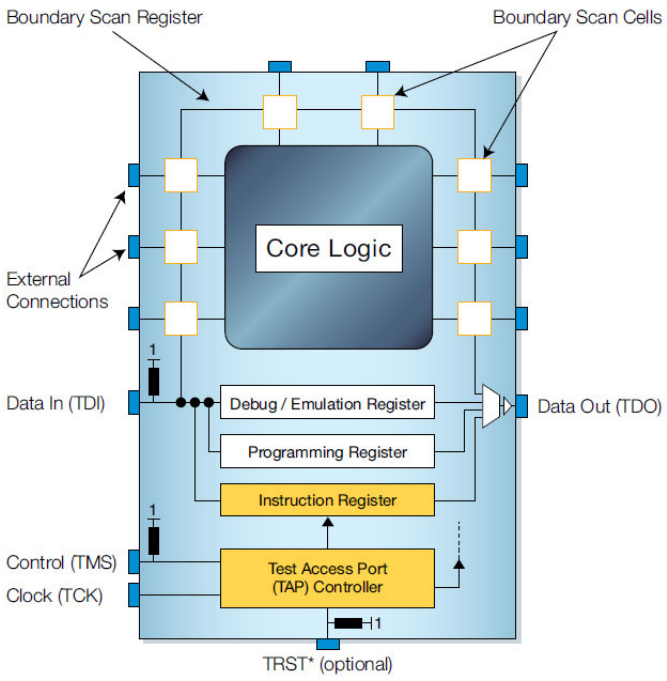

Figure 3.3.: JTAG interface in detail[4]

As shown in 3.3 it can be seen that that the JTAG device itself already contains some addition core logic for being able to perform different kind of test-debugging functions. The boundary scan register itself contains a various amount of boundary scan cells which could be used for the testing logic.

The TAP controller is a state machine which could take in six valid different states[5].

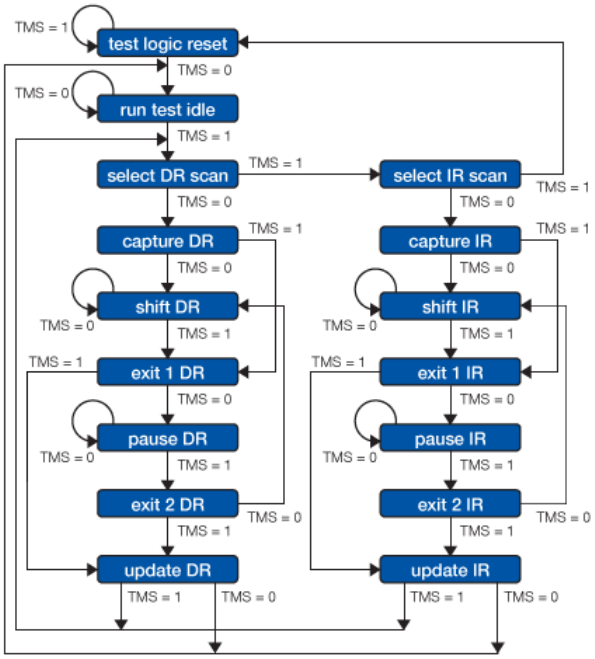

Figure 3.4.: State machine of the TAP controller[6]

As shown in 3.4 is will be clear that the transitions of each states are being controlled mainly by the TMS signal. Every state has two outgoings to make sure that just by one TMS step the condition could be set. The both main paths allow reading out data from the data register or the instruction register.

3.4. Boundary scan

The main idea of the boundary scan is the purpose of ensuring that every component is mounted correctly in physically way as described within the circuit diagram[7]. First of all according to IEEE 1149.1 an JTAG device needs to have the possibility of performing the following instructions to be a really boundary scan compatible device[6]:

- BYPASS - this instruction ensures that the TDI and TDO connections are connected by a one bit register and makes possible to save time during the operation
- EXTEST - this instruction makes sure that the TDI and TDO are connected with the boundary scan registers by providing functions to write values into the cells

- SAMPLE/PRELOAD - this instruction ensures that the TDI and TDO are connected with the boundary scan while operating in usual mode. within this mode the data could be read out

In detail that means that according to each external pin connection there is in general the possibility to set any value to high or low and read back the physically present value of wiring. So if it is known for example that on a specific pin position an according pull down resistor is being connected then the further knowledge is that in every case the read back value should be low. Additionally there can be assumed that if a pull up resistor is being connected to a pin independently on the set value the read back value should be high. Finally the fact can be accepted that if some other components are connected to any pin which are not belonging to the pull down nor the pull up resistors then the read back value should be that one which was previously set. To perform a complete test of a device the following steps must be followed:

1. The clock rate must be triggered as long as the complete boundary scan register length totally is[8]
2. During that time the TDI data will be forwarded into each boundary scan cell
3. If all cells are covered by a value then the TMS pin should be triggered for passing all read values into the connected pins
4. The clock rate must be triggered again for the total length of the boundary scan register
5. During that time all values from the external pins are being read out and transmitted into the TDO
6. Now the read out values could be compared with some expected values to detect differences

3.5. Boundary scan description language

The boundary scan description language is a language for describing the boundary scan test ability of different JTAG compatible devices. It is based on the very high speed integrated circuit hardware description language and normed with IEEE 1149.1-2001. Typically these information are being provided by the manufacturer of the devices in a .bsdl file for each component[9]. The files themselves are usually containing the following sections:

- port - the information about all pins and their usage mode (in, out, inout, linkage...)

- attribute PIN_MAP - the mapping from the used names within the port section to the real pin names used within the ODB++ project (position is important for later usage)
- attribute INSTRUCTION_LENGTH - instruction register length
- attribute BOUNDARY_REGISTER - all boundary scan compatible pins and their functionality (control, bidirectional, observe_only, output2)
- attribute BOUNDARY_LENGTH - the amount of boundary scan cells

3.6. SVF file format

The serial vector format is a format for communication with boundary scan compatible test vectors. It was introduced by Texas Instruments in 1991. The main purpose of the format is to ensure that independently on the used JTAG device different test operations could be performed successfully[10]. The format contains a couple of different instructions which could be performed by the boundary scan logic. The most important one according to this project is the **SDR length [TDI (tdi)] [TDO (tdo)][MASK (mask)] [SMASK (smask)];**. SDR means scan data register and tells the test to test all data register cells. The TDI is the data which should be passed into the registers[11]. TDO is the expected data to be read back from the data registers. And the MASK is the information about the positions of interest which should be considered. A valid instruction example is
SDR 24 TDI(000010) TDO(818181) MASK(FFFFFF);
In that specific example we have a total length of boundary scan cells of 24 and therefore the amount of passed hexadecimal coded byte values is 3 (24 bits / 8 bits/byte) -> 0x00 0x00 0x10 and 0x81 0x81 0x81 and 0xFF 0xFF 0xFF. Further the TDI value 000010_{16} means that only on position 5 the value should be set to high (10_{16} -> 00010000_2, counted from right to left). The TDO value 818181_{16} gives the information about the expected positions to be high after read back the data (818181_{16} -> $100000011000000110000001_2$). That signifies only the positions 1, 8, 9, 16, 17 and 24 (read from right to left) should be set to high as the needed result. Finally the MASK value FFFFFF $_{16}$ shows that all positions of the 24 bit register should be considered.

3.7. Model-view-viewModel

The Model-view-viewModel pattern came up with the introduction of the Windows presentation foundation (WPF) framework from Microsoft and is designed for loose coupling

programming with different components which are working together at the final stage. It could be seen as a enhancement of the model view controller pattern from JAVA programming language [12].

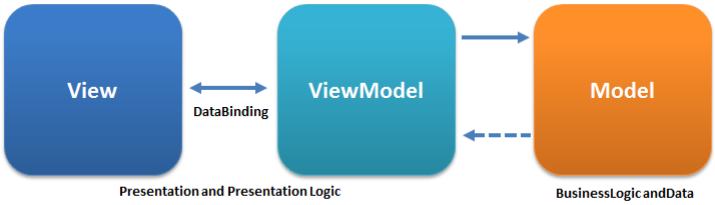

Figure 3.5.: MVVM structure[13]

As shown in 3.5 there are the three main components involved:

- View - the graphical representation form typically created in Extensible application markup language (XAML) code
- Model - content of all data to be used and business logic for it
- ViewModel - the connection layer to the view and contains representation logic

Typically there will be defined a various amount of data bindings in the view to the viewmodel. Further the view model itself informs the view about happened changes of values to be displayed by using the INotifyPropertyChanged event. Onward it is responsible for the connection to the data of any model and controls the function sequence to be performed in dependency on the user inputs which are being forwarded as ICommand objects. Finally the view should contain as less behind code as possible to make sure a high loose coupling is being ensured. The looser the coupling the easier it is to switch between new implementations of different components and replace them by others. In that way it should be quit easy possible to work on all three component types separately, without having to know of each others and finally connect them together.

3.8. PRISM framework

This framework is made for creation of louse coupled MVVM projects and allows to use predefined services as EventAggregator, the composite commands and dependency injection in a quit easy way[14]. Further it provides different project templates which could be used as start point of implementation.

3.9. Dependency injection framework

The **dependency injection framework** provides the possibility of registering interfaces by their implementations at some main points and being able to inject them directly in constructors of others classes without managing the creation of objects manually in background[15]. It can use further the **singleton pattern** for making sure just to work with a single instance of a class project wide.

3.10. Material design framework

The material design framework is a view orientated framework where the options are given to use many predefined components in a pretty and comfortable way[16]. Further it ensures a quit modern look and feel and will be used per default in many actual software implementations as in web design.

3.11. JSON format

The java script object notation is a serialized form of data output in textual form[17]. It is widely used where it could be necessary to understand and maybe modify data manually in an easy way. To work with that specific data format each programming language provides different kind of external libraries for parsing the information from objects into JSON and back from JSON to object. A valid JSON formatted text could like like:

```
{
  "Pins": [
    {
      "Nets": [
        0
      ],
      "Geometrics": {
        "Bounds": {
          "X": -1459,
          "Y": 286,
        },
        "UseRotation": true
      },
      "PinType": 2,
```

```
15         "PinNumber": "Pin1"
16       }
17     ]
18 }
```

Listing 3.1: JSON formatted content

As shown in 3.1 the format is highly structured and build in hierarchical way. Further it is clear that the curly brackets are being used for defining an object instance, the square brackets are being used for identifying an array of instances and the double quotes are describing a string value. With that quit simple format is is possible to represent just any Boolean, number or string value of objects where the requirement comes up either to use serializable or plain object structures.

3.12. GRPC

GRPC is a modern open source way for performing remote procedure calls independently on the programming language used for a specific program and works as a client-server structure[18]. That means in detail to use this protocol there is the need first of defining a specific service file using the interface definition language. By using this language there is no need to consider programming language specific dependencies. Afterward by using protoc compiler there is the possibility of automated creation of the source code for a specific programming language which could then be included to finalize the communication work. Finally on one side there should be defined the logic for being a server by additionally defining a specific port to be used and on the other there needs to be defined the logic for being the client with considering the IP address of the server and the port. Using GRPC ensures bidirectional streaming possibilities and passes all object related data as a auto serialized stream in the JSON format. Further it can communicate through complex networks without having the challenge of defining more complex code for it. To define the service file there are many of elementary data types available, the possibility of defining object structures, functions and parameters or return values. A valid example of such a definition could be:

```
1 syntax = "proto3";
2
3 service PcbInvestigatorService {
4   // Gets all objects
5   rpc GetConvertedObjects(Request) returns (Result) {};
6 }
7
8 message Request {
9    int32 requestNumber = 1;
```

```
10   ComponentTypeGrpc type = 2;
11 }
12
13 message Result {
14   bool status = 1;
15 }
16
17 enum ComponentTypeGrpc {
18   UnknownComponentType = 0;
19   Arc = 1;
20 }
```

Listing 3.2: Interface definition of GRPC exchange

As shown in 3.2 there are all necessary values defined in structured and easy understandable way. Typically the auto generated code contains new classes which must be inherited for adding customized logic.

Chapter 4.

Conceptual design

Within this chapter all thoughts and decisions which were made according the completion of the whole project are being pointed out in detail

4.1. Programming language to use

First of all there has been reflected which programming language to use for creating the new software. As there are many different languages available the following table shows the resulting advantages of two compared languages:

Programming language	Benefits	Disadvantages
C#	modern GUI applications possible, powerful, knowledge already available, already being used at ProMik	-
Java Swing	GUI applications possible	no loose coupling available, just little knowledge available, not modern and powerful
Java FX	modern GUI applications possible, powerful	no knowledge available, not being used at all at ProMik

Table 4.1.: Programming languages comparison

As stated in 4.1 it will be clear that the choice of the programming language to use finally turned into C# as the advantages convinced.

4.2. Integrated development environment to choose

Now there should be chosen an Integrated development environment (IDE) for creating the software with the previously concluded programming language as well:

IDE	Benefits	Disadvantages
JetBrains Rider	powerful	not for free available
Visual Studio Code	for free available	no nuget package manager available
Visual Studio 2019	powerful, for free available, already being used within ProMik	-

Table 4.2.: IDE comparison

As compared in 4.2 it sticks out that Visual Studio 2019 will be the choice for creating the new software as it's benefits are overbalanced.

4.3. Application structure to consider

Regarding the application structure there was the deliberation about how to use the PCB-Investigator API as efficient as possible without having the need of installing it on every machine which runs the new tool as there is the requirement of having a valid license for using it. The idea came up to provide on one side a stand alone application which uses the PCB-Investigator API as a server and on the other side the Graphical user interface (GUI) application as client software which is possible to connect to the server. In that case this server could potentially run just on one machine and would be available for all other clients which are in the same network.

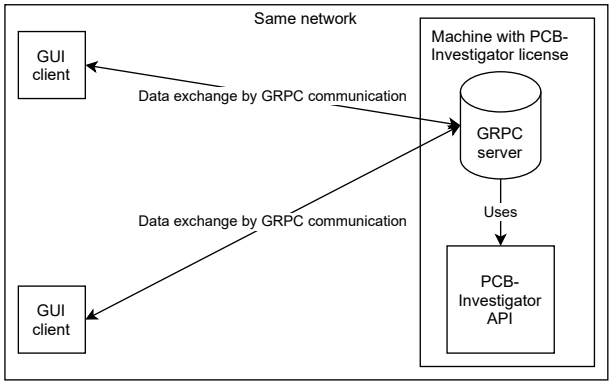

Figure 4.1.: Client-server structure of the planned software

As shown in 4.1 it is clear that following that idea a most flexible solution with as less restrictions as possible could be created. To make sure that the clients are able to communicate with the server the decision was made to use the GRPC protocol for data

exchange as it is already known within the company as a reliably protocol and could be used independently on the used programming languages or target frameworks of the projects.

4.4. Target frameworks to use

After the programming language, the IDE are selected and the application structure itself was defined, now there should be decided which target frameworks to use for all projects:

Framework	Benefits	Disadvantages
.NET framework	the only compatible one for using the PCB-Investigator API	not fully compatible with all provided nuget packages, no longer support
.NET standard	compatible to be included in .NET framework and .NET core applications	no graphical components available
.NET core	all modern graphical components available, all nuget packages compatible	graphical components included even if not used

Table 4.3.: Framework comparison

As shown in 4.3 each available framework has some advantages which makes it worth to use. So after thinking about it more in detail the following project structure came out to use:

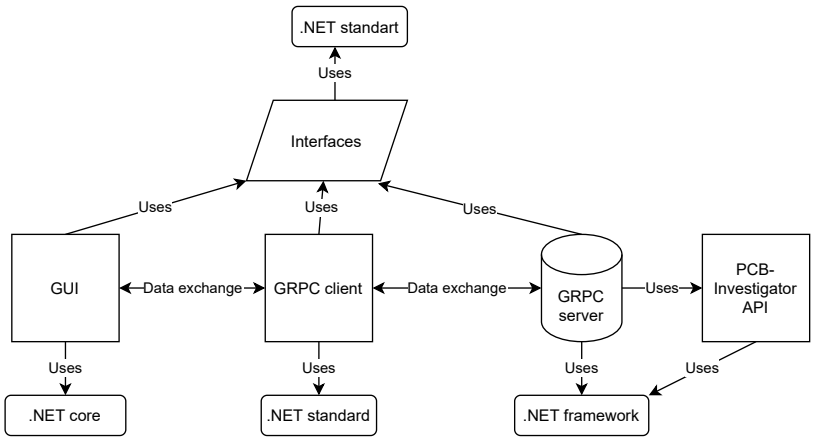

Figure 4.2.: Frameworks used within the project

The reason why to use .NET framework 4.72 for the server side is the fact that the provided API was created with that target framework (see 4.2). On the client side the reflection was considered to create a modern software as possible by using as much

useful frameworks for it as available and so the .NET core 3.1 has been chosen as target framework. To make sure finally that each project are able to use the common interfaces which should be created the target framework .NET standard 2.0 was used for it.

4.5. Analyzing the structure of PCB-Investigator API

Of course the structure of the PCB-Investigator API itself regarding the received objects needs to be analyzed more in detail which are being provided by the **PCB-Investigator.exe**:

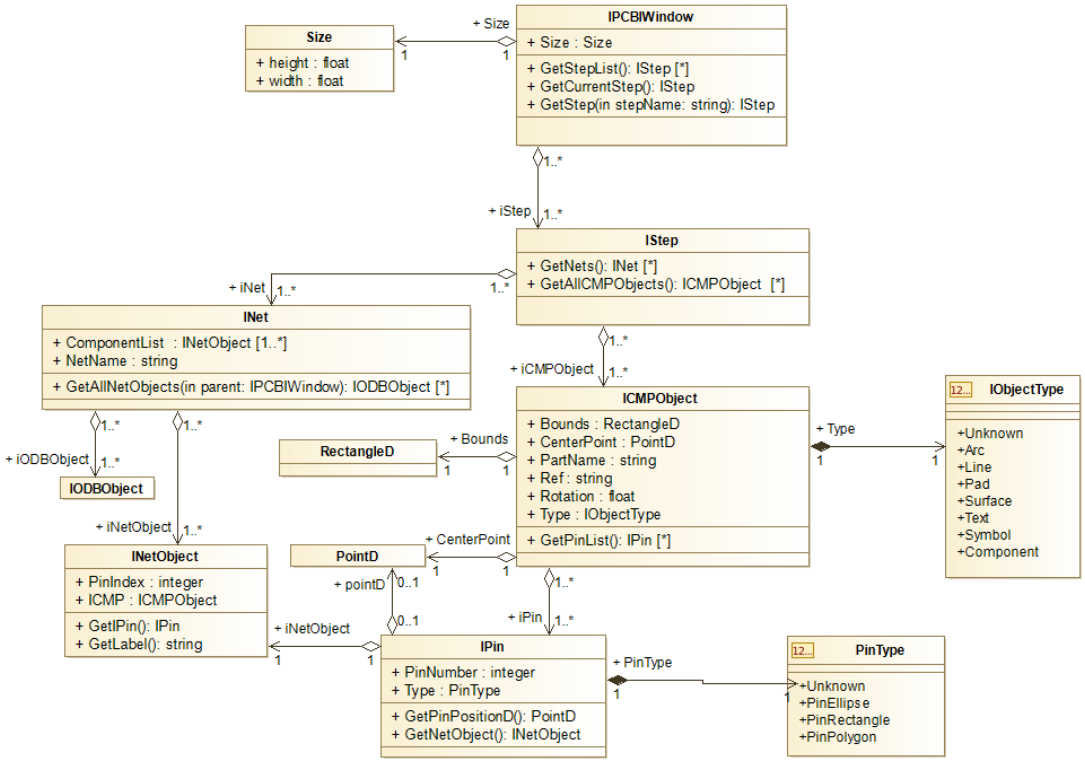

Figure 4.3.: Class diagram of the PCB-Investigator objects of interest

As shown within 4.3 there could be seen that the very first class is the IPCBIWindow which contains a various amount of IStep objects. Within this IStep objects there are many INet objects and ICMPObjects.
As ProMik GmbH is just interested in extracting all relevant information from any ODB++ related projects the following public functions of the **PCB-Investigator.exe** where used by the created software[19] to extract all INet and ICMPObjects:

- IAutomation.IAutomationInit() - Initialization of the Application interface (API)

- IAutomation.CreateNewPCBIWindow(bool visible) - Creates the base window
- IPCBIWindow.LoadData(string path) - Loads a specific project into the window
- IPCBIWindow.GetStepList() - Getter for all steps of the window
- IStep.GetNets() - Getter for all nets of a step
- IStep.GetAllLayerNames() - Getter for all layer names of a step
- IStep.GetLayer(string layerName) - Getter for a layer in dependency on the layer name from a step
- ILayer.GetAllLayerObjects() - Getter for all objects which are located on a specific layer within a step

4.6. Converted class structures of objects of interest

After knowing what kind of functions to be used and which structures are available within the PCB-Investigator API the goal was to provide as simple new structures as possible containing all relevant information for later usage:

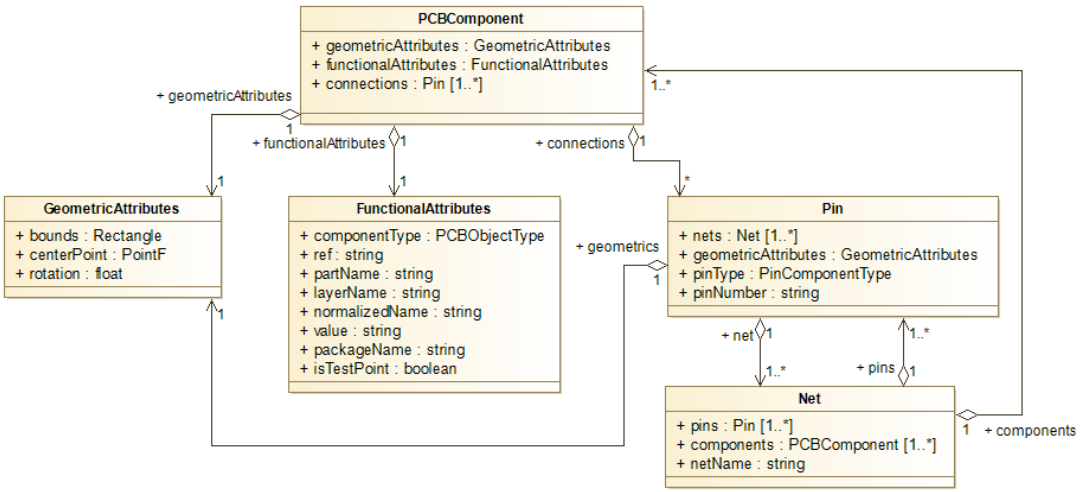

Figure 4.4.: Class structure after convertion

As shown in 4.4 it will be clear that the new object structure just contains five main components

- PCBComponent - the main component of interest containing further objects with information about their geometric, functional attributes and connection pins
- Pin - the connection component of a PCBComponent which contains net objects and information about the pin name and its type
- Net - the net component containing the net name, information about all pins and components which are included
- GeometricAttributes - an object for holding all information regarding the position and size of any object
- FunctionalAttributes - an object for having all information regarding the functionality of one component (type and names)

To make sure for having a highest level on flexibility of course as top level objects there should be interfaces introduced which then will be implemented for later usage.

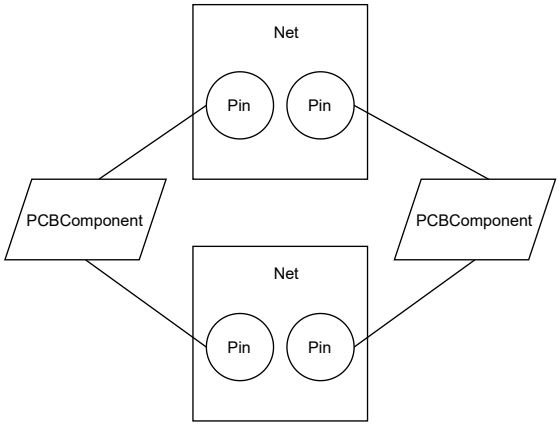

Figure 4.5.: Physical object dependencies

As shown in 4.5 it is possible to map all physical existing objects easily in complete way into the object orientation which is used by the programming language by using this class structure.

4.7. Frameworks and patterns to use

To enure a modern, loose coupled appearance of the whole application where it is quit easy possible to build in different kind of extensions there was made the decision of using the **PRISM framework** for referencing a template as an empty new project.

Within that framework already the possibility of using the **dependency injection framework** is given which should be used as well to make sure to be flexible in changing the implementations used for different interfaces. Further PRISM makes possible to use the modern appearance style from the **material design framework** which should be used as well to make sure the software looks modern and pretty. To ensure creation of different implementations of interfaces within just one method[20] there was additionally the need of using the **factory pattern** at some points. Next there should be used the **command pattern** to make sure that a loose coupling between views and view-models could be established without any code behind in the views. Finally for being able to work with as many interfaces as possible to ensure easy change of implementations there should be used the **strategy pattern** which exactly means that there are many implementations of interfaces available[21].

Chapter 5.

Architecture

5.1. Overview

5.2. Interfaces

5.3. Grpc Server for using the PCB Investigator API

5.4. Grpc Client for connecting to the Server and parsing all objects

5.5. GUI

5.5.1. Events

5.5.2. Helper

5.5.3. Interfaces

5.5.4. Implementations

5.5.5. Model

5.5.6. ViewModels

5.5.7. Views

5.5.8. MainView and MainViewModel

5.6. Test coverage determination of loaded project

5.7. Pin information extraction

5.8. SVF file generator

5.9. Unit tests

Chapter 6.

Analysis

6.1. Performance of loading ODB++ projects by using the PCBInvestigator API

6.2. Performance of loading projects by importing JSON formatted files

6.3. Performance of exporting projects into JSON format

6.4. Performance of showing connections of components

6.5. Performance of determination of test coverage values

Chapter 7.

Conclusion

Chapter 8.

Thanksgiving

Within this small chapter I would like to thank a couple of people who made this work possible by supporting me in many different areas of my life.
First of all I want to thank my parents **Gabriela** and **Nicolae Matis** who have born me and supported me in every single minute of my life. I know I unfortunately didn't always made you proud of my actions but I hope that I am still working on it successfully by milestones like this one. Mum I really hope for you that you will solve your physical constraints successfully and dad I hope that you stay as young as you are for a really long time

Afterward I would like to thank my brother **Nik Matis** and my sister **Michelle Matis** who have supported me in that way as a brother just could expect it to be perfect. Michelle I wish you all the best with your study and Nik I hope you will find your destiny in choosing a new job soon.

And finally I would like to thank a couple of my best friends who have supported me in investing much free time with myself and were successfully responsible for being able to forget all the stress about the work and enjoy my life.

Andreas Korkenländer I already know you now about for 20 years and really hope that this will continue until my death. I wish you all the best with your new job and your intention of traveling around the world.
Dominik Müller you were my first best friend after I moved to Nürnberg and I really hope that connection persists for a longer time although you are sometimes really a hardheaded person :) I wish you all the best with your new challenge of doing a new apprenticeship.
Rene Biedermann you are one of that guys I'm able to speak about philosophic and esoteric topics and of course medical topics about my own health in a detailed way as I'm really impressed of it. I wish you all the best in accomplishing your exam for being

an alternative practitioner and I really think that this will satisfy your life in complete way.

Willi Fritz I would really like to thank you especially for the last year where you permitted me to do my training within your own flat and visited me almost every weekend for just hanging around and playing PlayStation 4 or for cooking some real tasty meals. I wish you all the best at finishing your tax consultant state examination and am really looking forward of traveling around the world with your person soon.

Appendix A.

Appendix

List of Figures

3.1. ODB++ project structure[2] 6
3.2. PCB-Investigator overview 7
3.3. JTAG interface in detail[4] 8
3.4. State machine of the TAP controller[6] 9
3.5. MVVM structure[13] 12

4.1. Client-server structure of the planned software 17
4.2. Frameworks used within the project 18
4.3. Class diagram of the PCB-Investigator objects of interest 19
4.4. Class structure after convertion 20
4.5. Physical object dependencies 21

List of Tables

4.1. Programming languages comparison 16
4.2. IDE comparison 17
4.3. Framework comparison 18

List of Listings

3.1. JSON formatted content 13
3.2. Interface definition of GRPC exchange 14

Bibliography

[1] ODB++Design, "Why odb++design - odb++design," 5/18/2020. [Online]. Available: https://odbplusplus.com/design/why-odb-d/

[2] "What is odb++," 2/5/2019. [Online]. Available: https://www.artwork.com/odb++/odb++_overview.htm

[3] "Pcb-investigator," 4/7/2021. [Online]. Available: https://www.pcb-investigator.com/de/functions

[4] XJTAG, "Was ist jtag und wie kann ich es nutzen? - xjtag tutorial," 8/27/2020. [Online]. Available: https://www.xjtag.com/de/about-jtag/what-is-jtag/

[5] "Joint test action group – wikipedia," 3/25/2021. [Online]. Available: https://de.wikipedia.org/wiki/Joint_Test_Action_Group

[6] XJTAG, "Technischer leitfaden für jtag boundary-scan - xjtag tutorial," 13.03.2020. [Online]. Available: https://www.xjtag.com/de/about-jtag/jtag-a-technical-overview/

[7] Harry Bleeker, *Boundary-Scan Test: A Practical Approach*. Springer Science+Business Media Dordrecht, 1993.

[8] R.C. Cofer and Benjamin F. Harding, *Rapid System Ptototyping with FPGAs*. Elsevier Inc., 2006.

[9] "Boundary scan description language – wikipedia," 3/25/2021. [Online]. Available: https://de.wikipedia.org/wiki/Boundary_Scan_Description_Language

[10] "Serial vector format – wikipedia," 3/25/2021. [Online]. Available: https://de.wikipedia.org/wiki/Serial_Vector_Format

[11] TEXAS INSTRUMENTS, ASSET InterTech, Inc., *SVF SERIAL VECTOR FORMAT SPECIFICATION JTAGBOUNDARY SCAN*, 2013.

[12] Norbert Eder, "Mvvm: Das viewmodel," 2010. [Online]. Available: https://www.norberteder.com/MVVM-Das-ViewModel/

[13] "Model–view–viewmodel," 2021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Modelviewviewmodel&oldid=1012221294

[14] “Introduction to prism | prism,” 4/5/2021. [Online]. Available: https://prismlibrary.com/docs/

[15] S. Augsten, “Was ist dependency injection?” 4/9/2021. [Online]. Available: https://www.dev-insider.de/was-ist-dependency-injection-a-814452/

[16] “Components - material design,” 1/1/1980. [Online]. Available: https://material.io/components

[17] “Json,” 29.03.2021. [Online]. Available: https://www.json.org/json-en.html

[18] gRPC, “"><meta property,” 4/8/2021. [Online]. Available: https://grpc.io/

[19] “Pcb-investigator interface documentation - table of content,” 4/7/2021. [Online]. Available: https://manual.pcb-investigator.com/InterfaceDocumentation/index.php

[20] JAXenter, “Mit factory-pattern designprobleme lösen - jaxenter,” 2012. [Online]. Available: https://jaxenter.de/mit-factory-pattern-designprobleme-losen-5040

[21] “Strategie (entwurfsmuster),” 2021. [Online]. Available: https://de.wikipedia.org/w/index.php?title=Strategie_(Entwurfsmuster)&oldid=207453256